Operating System CSC3150 Project 4

Yuqi Ma

yuqima1@link.cuhk.edu.cn

Content

- Some **hints** about the project
  - sys_mmap()
  - trap(page fault handling)
  - sys_munmap()
  - extra credits
- Q&A

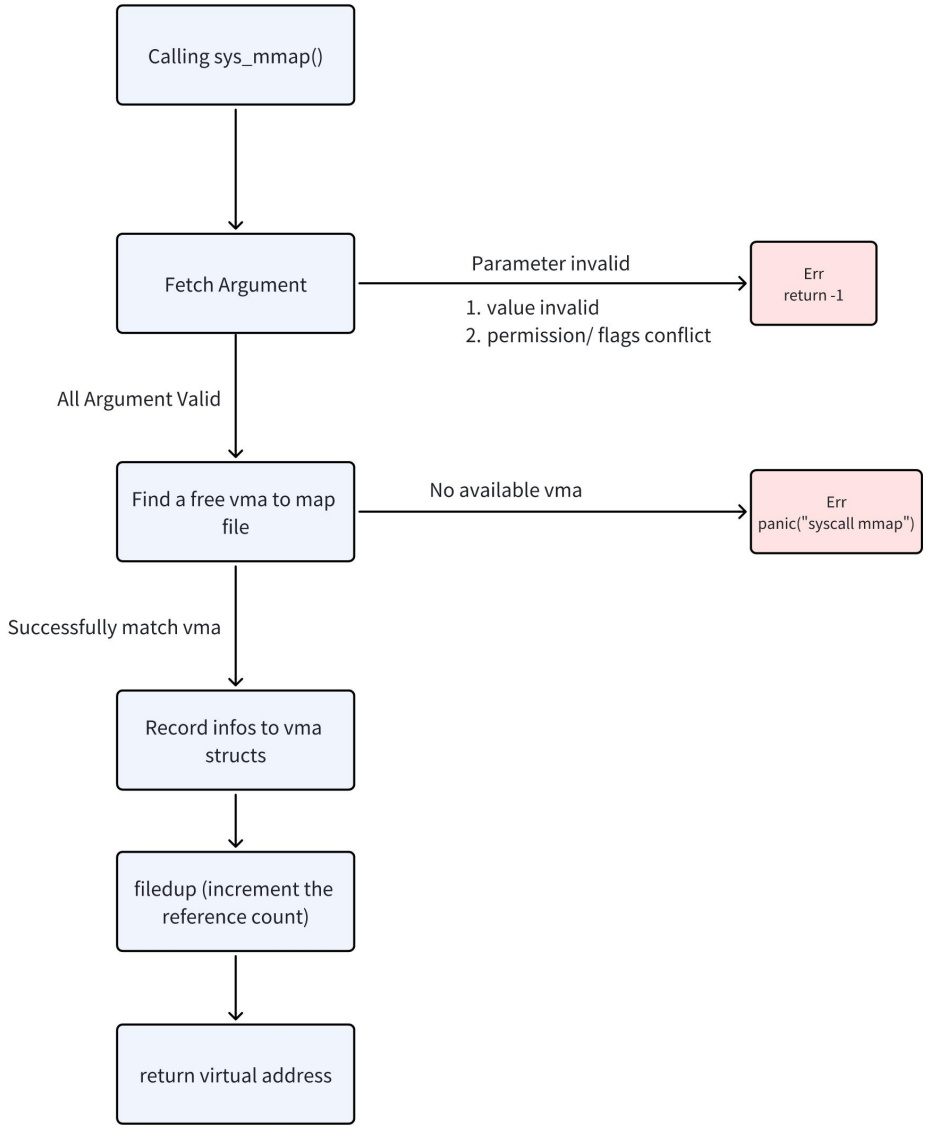
sys_mmap()

- Review of previous tutorial

  1. Fetch the arguments and check if they are valid from the user call.

  2. Search for an available VMA in the current process and record the map information.

  3. Increase the file duplicate.



sys_mmap()

Fetch the arguments and check if they are valid from the user call.

- Step 1.

Use argxxx() to fetch all arguments from the trapframe.

Check if they are valid from the user call.

You may check whether addr equals to 0? length, prot, flags, fd, offset should be in a reasonable range.

f->readable should not conflict with prot(non-readable and non-writable file check)

sys_mmap()

Find an available address and VMA.

- Step 2.

Hints about how to find an available address:

Refer to the userspace struct in memlayout.h

The topmost address you can use is under the TRAPFRAME.

You can search the available empty space according to the VMA information. Meanwhile, find an VMA with valid bit e to fill the information.

sys_mmap()

Increase the file duplicate

- Step 3.

Use filedup() to increment the reference count of the file and return the starting address

Page Fault Handling

- Review of previous tutorial

1. Locate the VMA using the fault address.

2. Read 4096 bytes (1 PAGESIZE) of the relevant file onto that page, map it into the user address space.

3. Set the permissions correctly on the page.

Page Fault Handling

Locate the VMA using the fault address.

- Step 1.

use r_stval() to get the faulting virtual address

Find which VMA this address belong to. The easiest way is to traverse all the VMAs.

Page Fault Handling

Read 4096 bytes (1 PAGESIZE) of the relevant file onto that page

- Step 2.

Read PGSIZE bytes of the relevant file into that page, and map it into the user address space
You should try to allocate the physical page use kalloc() to allocaate a page
Then read the file into target physical page mapfile() may be helpful

Page Fault Handling

Map the page and set permission

- Step 3.

Set the correct permission and map physical page and virtual page into pagetable.
You should set PTE flags according to the VMA prot. use mappages()

sys_munmap()

- Review of previous tutorial

1. Find the VMA for the address range and unmap the specified pages. (use `uvmunmap`)
2. If munmap removes all pages of a previous mmap, it should decrease the reference count of the corresponding struct file.
3. If an unmapped page has been modified and the file is mapped MAP_SHARED, write the page back to the file. (use `filewrite`)

```
/*
step 1.
Fetch arguments

Find corresponding VMA, similar to the first step of page fault handling.

Change the address range of VMA.
*/

/*
step 2.
Write back to the file if MAP_SHARED is set. filewrite() may be useful.

Remove the mapping. Using uvmunmap().
*/

/*
step 3.
Decrement the reference count of the file if none is left.
You may use fileclose() in this step.
Then free the VMA.
*/
```

Extra Credits

Hints:

Read fork() and exit() in proc.c

Key point is the transfer of mapping information.

Some of them have already been completed.

Find the missing part(You can refer the struct of proc) and copy from parent to the child in fork()

Remove mapping in exit().

Q&A